

Laboratorio 04 – UART/USART

Instrucciones

Este laboratorio se realiza en grupos de 3 personas durante la semana 08 del curso. Al finalizar adjunta en Canvas la versión **.pdf** con las respuestas a las preguntas, los archivos **.c** con los códigos correspondientes a los archivos **main.c** de los ejercicios de programación, y los enlaces de **YouTube** en donde se solicitan videos.

Objetivo

Programar la placa STM32 Nucleo de forma directa sobre el hardware, sin usar un sistema operativo ni capas de abstracción de alto nivel. Se introduce el periférico **USART2** para lograr comunicación serial con una computadora mediante el software **RealTerm**, manteniendo el enfoque de registros (CMSIS) aprendido en el Lab 03.

PARTE 01

1) Preparación del proyecto (continuación Lab 03)

Duplica el proyecto del Lab 03 (Project Explorer → click derecho → Copy/Paste o Duplicate según tu flujo).

Renombra el proyecto como: Lab04_UART.

La idea es que el proyecto ya tenga: includes CMSIS, símbolo STM32F446xx y estructura de compilación funcional (lo ya trabajado en el Lab 03).

2) Archivos base que deben incluirse en el proyecto (plantillas adjuntas a esta guía)

Dentro del proyecto (carpetas típicas Core/Src y Core/Inc), incluir:

Obligatorios:

uart.h = uart_lab04.h (en Core/Inc)

```
#ifndef UART_LAB04_H
#define UART_LAB04_H

#include <stdint.h>

/*
 * Driver UART/USART2 (STM32F446) - CMSIS / Registros
 * Placa: NUCLEO-F446RE
 * USART2: PA2 = TX, PA3 = RX, AF7
 */

void uart2_init(uint32_t baud, uint8_t enable_rx);
void uart2_write_char(char c);
void uart2_write_str(const char *s);

/* RX (polling) */
char uart2_read_char_blocking(void); /* espera RXNE */

/* Utilidad: enviar número decimal por UART (opcional) */
void uart2_write_u32(uint32_t v);

#endif
```

uart.c = uart_lab04.c (en Core/Src)

```
#include "uart_lab04.h"
#include "stm32f4xx.h"

#ifndef CPU_HZ
#define CPU_HZ 16000000UL /* Ajustar si el reloj no es 16 MHz en su proyecto */
#endif

static uint32_t compute_brr(uint32_t baud)
{
    /* Aproximación simple: BRR = fCK / baud (redondeo) */
    return (CPU_HZ + (baud/2U)) / baud;
}

void uart2_init(uint32_t baud, uint8_t enable_rx)
{
    /* 1) Clocks: GPIOA y USART2 */
    RCC->AHB1ENR |= RCC_AHB1ENR_GPIOAEN;
    RCC->APB1ENR |= RCC_APB1ENR_USART2EN;

    /* 2) PA2 y PA3 -> Alternate Function (MODER=10) */
    GPIOA->MODER &= ~((3U << (2U*2U)) | (3U << (3U*2U)));
    GPIOA->MODER |= ((2U << (2U*2U)) | (2U << (3U*2U)));

    /* 3) AF7 en PA2 y PA3 (AFRL) */
    GPIOA->AFR[0] &= ~((0xFU << (4U*2U)) | (0xFU << (4U*3U)));
    GPIOA->AFR[0] |= ((7U << (4U*2U)) | (7U << (4U*3U)));

    /* 4) USART2: 8N1 por defecto, baud en BRR */
    USART2->CR1 = 0;
    USART2->BRR = compute_brr(baud);

    /* 5) Habilitar TX y (opcional) RX */
    USART2->CR1 |= USART_CR1_TE;
    if (enable_rx) USART2->CR1 |= USART_CR1_RE;

    /* 6) Enable USART */
    USART2->CR1 |= USART_CR1_UE;
}

void uart2_write_char(char c)
{
    while ((USART2->SR & USART_SR_TXE) == 0) {}
    USART2->DR = (uint8_t)c;
}
```

Plantillas de main:

main_ex1_baud_bench_lab04.c

```

#define LED_PIN (1U << 5) /* LD2 en PA5 */

static void delay(volatile uint32_t n) { while(n--) {} }

static void gpio_led_init(void)
{
    RCC->AHB1ENR |= RCC_AHB1ENR_GPIOAEN;
    GPIOA->MODER &= ~(3U << (5U*2U));
    GPIOA->MODER |= (1U << (5U*2U));
}

static void led_toggle(void) { GPIOA->ODR ^= LED_PIN; }

static void send_frame(const char* payload)
{
    const uint8_t STX = 0x02, ETX = 0x03;
    uint8_t len = 0;

    while (payload[len] && len < 50) len++;

    uart2_write_char((char)STX);
    uart2_write_char((char)len);

    uint8_t chk = len;
    for (uint8_t i = 0; i < len; i++) {
        uart2_write_char(payload[i]);
        chk ^= (uint8_t)payload[i];
    }
    uart2_write_char((char)chk);
    uart2_write_char((char)ETX);
    uart2_write_str("\r\n");
}

int main(void)
{
    gpio_led_init();

    /* TODO: Cambiar baud para el bench: 9600/19200/38400/115200 */
    const uint32_t baud = 115200;
    uart2_init(baud, 0);

```

main_ex2_rx_terminal_lab04.c

```
#define LED_PIN    (1U << 5) /* LD2 PA5 */

static void delay(volatile uint32_t n) { while(n--) {} }

static void gpio_led_init(void)
{
    RCC->AHB1ENR |= RCC_AHB1ENR_GPIOAEN;
    GPIOA->MODER &= ~(3U << (5U*2U));
    GPIOA->MODER |= (1U << (5U*2U));
}

static void led_on(void) { GPIOA->BSRR = LED_PIN; }
static void led_off(void) { GPIOA->BSRR = (LED_PIN << 16); }

static void led_blink_times(uint32_t n)
{
    for (uint32_t i = 0; i < n; i++) {
        led_on(); delay(600000);
        led_off(); delay(600000);
    }
}

static uint32_t str_len(const char* s)
{
    uint32_t n = 0;
    while (s && s[n]) n++;
    return n;
}

static int str_eq(const char* a, const char* b)
{
    uint32_t i = 0;
    while (a[i] && b[i]) {
        if (a[i] != b[i]) return 0;
        i++;
    }
    return (a[i] == '\0' && b[i] == '\0');
}

int main(void)
```

Archivos solo de referencia, no se entregan como main final:

main_printf_no_modular.c (sirve como comparación “NO modular”, opcional)

Se debe considerar que el **main.c no conozca demasiado hardware**, y que UART viva en uart.c/uart.h como driver reusable (modularidad).

3) Configuración de RealTerm (PC)

Los estudiantes deben:

- Identificar el **COM** del ST-LINK (Administrador de dispositivos).
- Abrir **RealTerm** y configurar:
 - Puerto COM correcto
 - Baud rate inicial: **115200** (luego variará en el bench)
 - 8N1 (8 bits, sin paridad, 1 stop)

- Activar “Open” para abrir el puerto.

Evidencia para el video: debe verse RealTerm recibiendo y/o enviando datos en tiempo real.

PARTE 02

Ejercicio 1: Bench experimental de Baud Rate (con y sin protocolo)

Objetivo

Realizar un bench comparativo de comunicación UART variando tasas de baudios y comparando:

Modo A (sin protocolo): envío de datos “en crudo”

Modo B (con protocolo simple): datos encapsulados con inicio/fin y verificación

Requerimientos del driver (uart.c/uart.h)

Modificar el driver para permitir:

Inicialización con baud variable (ejemplo de interfaz recomendada):

- void uart2_init(uint32_t baud);
- void uart2_write_char(char c);
- void uart2_write_str(const char* s);

El objetivo es que el **mismo driver** soporte múltiples configuraciones, y el main_ex1_baud_bench_lab04.c solo llame funciones.

Parte 2.1 – Prueba sin protocolo (raw stream)

En main_ex1_baud_bench_lab04.c implementar:

Configurar uart2_init(baud) con cada uno de estos baudios:

- 9600
- 19200
- 38400
- 115200

Para cada baud:

Enviar por UART una secuencia repetitiva fácil de identificar, por ejemplo:

- 0123456789ABCDEF\r\n

Mantener transmisión cada 200 ms (o 500 ms) para observar estabilidad.

Probar también el caso de error:

- Firmware en 115200, RealTerm en 9600 (y viceversa)

Observar y describir por qué ocurre.

Parte 2.2 – Prueba con protocolo simple (framing)

Implementar un protocolo mínimo para transmitir “frames”:

Formato sugerido:

STX = 0x02

LEN = longitud del payload

PAYLOAD

CHK = XOR (LEN y bytes del payload)

ETX = 0x03

Para cada baud rate:

- Enviar un frame con contador, Payload: "CNT=000123"

Preguntas (se responden en PDF)

1. ¿Cómo ayuda el framing cuando hay error?
2. ¿Se detecta corrupción por checksum?
3. ¿Qué diferencia hay entre **baud rate** y **bit rate**?
4. ¿Qué registro se usa para configurar baud rate en USART y qué representa su valor?
5. ¿Qué se observa en RealTerm cuando el baud del PC no coincide con el baud del micro?
6. ¿Qué ventaja aporta el protocolo (STX/LEN/CHK/ETX) frente al modo sin protocolo?

Video obligatorio – Ejercicio 1 (YouTube)

En el video debe verse:

- RealTerm recibiendo datos a **2 baud rates distintos**
- Un caso “malo” (baud mismatch) mostrando caracteres corruptos
- Un caso “con protocolo” y explicación de qué detecta el checksum/framing

PARTE 03

Ejercicio 2: Terminal UART (RX), recepción de cadenas y acción con LED

Objetivo

Permitir que el usuario envíe una línea de texto desde RealTerm al STM32. La placa recibirá la cadena y ejecutará una acción con el LED LD2 según el contenido.

Requerimientos del driver (uart.c/uart.h)

Agregar soporte RX (polling):

- char uart2_read_char(void); (bloqueante por RXNE)
- (opcional) int uart2_try_read_char(char* out); (no bloqueante)

Parte 3.1 – Recepción de línea (buffer)

En main_ex2_rx_terminal_lab04.c:

Declarar un buffer:

- char buf[64];
- int idx = 0;

Recibir caracteres desde UART:

- Guardar cada char en buf[idx++]
- Si llega '\r' o '\n':
- terminar cadena con '\0'
- procesar cadena
- resetear idx = 0

Control de overflow:

- Si idx == 63, resetear y enviar mensaje de error por UART
- "ERROR: overflow\r\n"

Parte 3.2 – Regla principal

Al recibir una cadena completa:

El LED LD2 debe parpadear N veces, donde **N = longitud de la cadena** (sin contar \r\n)

Ejemplos:

- "Hola" → 4 parpadeos

- "UVG" → 3 parpadeos

Parte 3.3 – Comandos simples

Además de la regla de longitud, implementar al menos **2 comandos**:

- ON → LED encendido fijo
- OFF → LED apagado fijo
- LEN → responder por UART el número de caracteres de la última cadena

Preguntas (se responden en PDF)

- ¿Qué bandera indica que un dato fue recibido (RX) en USART?
- ¿Por qué es importante manejar overflow en el buffer?
- ¿Qué diferencia hay entre leer carácter a carácter y procesar por línea completa?

Video obligatorio – Ejercicio 2 (YouTube)

El video debe mostrar:

- Envío de al menos 3 cadenas desde RealTerm
- Parpadeos correctos según longitud y caso
- Ejecución de comandos (ON/OFF/LEN)
- Explicación breve del buffer y fin de línea

Entregables finales

PDF con:

- Respuestas a preguntas (Ej. 1 y Ej. 2)
- Tabla de resultados del bench (baud vs comportamiento)

Archivos .c:

- uart.c, uart.h
- main_ex1_baud_bench_lab04.c
- main_ex2_rx_terminal_lab04.c

Links de YouTube:

1 enlace de video con el Ejercicio 1 y Ejercicio 2

